

Hands-on Activity 12.1		
Algorithmic Strategies		
Course Code: CPE010	Program: Computer Engineering	
Course Title: Data Structures and Algorithms	Date Performed: Oct 25, 2025	
Section: CPE21S4	Date Submitted: Oct 25, 2025	
Name(s): Punay, Heidee S.	Instructor: Engr. Jimlord Quejado	
A. Output(s) and Observation(s)		
ILO A: Demonstrate an understanding of algorithmic strategies (Pre-Lab Review)		
Algorithmic Strategies and Examples		
Strategy	Algorithm	Analysis
Recursion	Recursive Fibonacci	This algorithm calls itself twice until it reaches the base cases. It demonstrates recursion because the problem is divided into smaller subproblems of the same type.
Brute Force	Linear Search	Linear Search checks each element of a list one by one until it finds the target value or reaches the end. It starts at the first element of the list and compares it with the target value. Next, it moves to the next element if it's not a match, it repeats until it finds the target or reaches the end.
Backtracking	Depth First Search	Depth-first search uses backtracking; it visits all neighbor nodes and checks if there are still nodes, and backtracks if there are no more neighbor nodes.
Greedy	Binary Search	Binary Search repeats to select the middle element as a choice to reduce the search space by half each time. Greedy because it always picks the midpoint as the next step without revisiting earlier elements.
Divide-and-conquer	Merge Sort	Merge sort divides the array into halves, recursively sorts each half, and then merges the sorted halves. Merge sort demonstrates a divide-and-conquer strategy.

Memoization Implementation

Screenshot

main.cpp

```
1  #include <iostream>
2  #include <cstring>
3  #include <algorithm>
4  using namespace std;
5
6  int memo[1000];
7
8  int getMinSteps(int n) {
9      if (n == 1)
10         return 0;
11     if (memo[n] != -1)
12         return memo[n];
13     int r = 1 + getMinSteps(n - 1);
14     if (n % 2 == 0)
15         r = min(r, 1 + getMinSteps(n / 2));
16     if (n % 3 == 0)
17         r = min(r, 1 + getMinSteps(n / 3));
18     memo[n] = r;
19     return r;
20 }
21
22 int main() {
23     memset(memo, -1, sizeof(memo));
24     int n;
25     cout << "Enter a number: ";
26     cin >> n;
27     cout << "Minimum steps to reduce " << n << " to 1 = " << getMinSteps(n) << endl;
28
29     return 0;
30 }
```

OUTPUT:**Output**

```
Enter a number: 10
Minimum steps to reduce 10 to 1 = 3
```

```
=== Code Execution Successful ===
```

Analysis

This program finds the minimum number of steps to reduce a number n to 1. To reduce this, it can either subtract it by 1, divide it by 2, and divide it by 3. To find the smallest number of steps, it uses a function called `getMinSteps(n)`. First, it checks if the number is 1, and there will be no more steps needed, so the function returns 0. Next, it checks if it already knows the answer for this number. If n is not equal to -1, it returns the saved answer instead of solving again. Next, it tries the first operation, which is subtracting 1 from n , and it also checks if the number is divisible by 2 or 3.

Bottom-Up Dynamic Programming Implementation**Screenshot**

main.cpp

```
1  #include <iostream>
2  #include <algorithm>
3  using namespace std;
4
5  int getMinSteps(int n) {
6      int dp[n + 1];
7
8      for (int i = 2; i <= n; i++) {
9          dp[i] = 1 + dp[i - 1];
10         if (i % 2 == 0)
11             dp[i] = min(dp[i], 1 + dp[i / 2]);
12         if (i % 3 == 0)
13             dp[i] = min(dp[i], 1 + dp[i / 3]);
14     }
15
16     return dp[n];
17 }
18
19 int main() {
20     int n;
21     cout << "Enter a number: ";
22     cin >> n;
23
24     cout << "Minimum steps to reduce " << n << " to 1 = " << getMinSteps(n) << endl;
25
26     return 0;
27 }
28
```

OUTPUT:

	Output <pre> Enter a number: 10 Minimum steps to reduce 10 to 1 = 3 === Code Execution Successful === </pre>
Analysis	This program calculates the minimum number of steps needed to reduce a number to 1. It can also subtract by 1, or divide it by 2 if it is divisible, and 3 if it is also divisible by 3. This program finds the smallest number of steps to reach 1. Bottom-up dynamic programming starts from the simple case and builds the solution step by step until it reaches n.

B. Answers to Supplementary Activity

Pseudocode

```

FUNCTION countPaths(cost, m, n, targetCost)
  IF (m == 0 AND n == 0)
    IF targetCost == cost[0][0]
      RETURN 1
    ELSE
      RETURN 0
  END IF

  IF targetCost < 0
    RETURN 0

  paths = 0

  IF m > 0
    paths = paths + countPaths(cost, m - 1, n, targetCost - cost[m - 1][n])
  END IF

  IF n > 0
    paths = paths + countPaths(cost, m, n - 1, targetCost - cost[m][n - 1])
  END IF

  RETURN paths
END FUNCTION

MAIN:
  INPUT M, N, targetCost
  INPUT cost matrix of size M×N
  totalPaths = countPaths(cost, M - 1, N - 1, targetCost - cost[M - 1][N - 1])
  PRINT totalPaths
END MAIN

```

--



```
1  #include <iostream>
2  using namespace std;
3
4  int countPaths(int **cost, int m, int n, int targetCost) {
5      if (m == 0 && n == 0)
6          return (targetCost == cost[0][0]) ? 1 : 0;
7
8      if (targetCost < 0)
9          return 0;
10
11     int paths = 0;
12
13     if (m > 0)
14         paths += countPaths(cost, m - 1, n, targetCost - cost[m - 1][n]);
15
16     if (n > 0)
17         paths += countPaths(cost, m, n - 1, targetCost - cost[m][n - 1]);
18
19     return paths;
20 }
21
22 int main() {
23     int M, N, targetCost;
24
25     cout << "Enter number of rows (M): ";
26     cin >> M;
27     cout << "Enter number of columns (N): ";
28     cin >> N;
29
30     // Create dynamic matrix
31     int **cost = new int*[M];
32     for (int i = 0; i < M; i++)
```

```

33     cost[i] = new int[N];
34
35     cout << "Enter the cost matrix (" << M << "x" << N << "):\n";
36     for (int i = 0; i < M; i++)
37         for (int j = 0; j < N; j++)
38             cin >> cost[i][j];
39
40     cout << "Enter target cost: ";
41     cin >> targetCost;
42
43     int totalPaths = countPaths(cost, M - 1, N - 1, targetCost - cost[M - 1][N - 1]);
44
45     cout << "\nNumber of paths with total cost " << targetCost << " = " << totalPaths <<
        endl;
46
47     for (int i = 0; i < M; i++)
48         delete[] cost[i];
49     delete[] cost;
50
51     return 0;

```

OUTPUT:

Output

```

Enter number of rows (M): 4
Enter number of columns (N): 4
Enter the cost matrix (4x4):
4 7 1 6
6 7 3 9
3 8 1 2
7 1 7 3
Enter target cost: 25

Number of paths with total cost 25 = 1

=== Code Execution Successful ===

```

C. Conclusion & Lessons Learned

To conclude, through this activity, I understood how the algorithms could be used for different strategies. It made me learn again and recall the different strategies and algorithms, and how they function. And also to know the difference between memoization and bottom-up dynamic programming.

In the supplementary activity, this program uses recursion to count how many paths in a matrix from the top-left to the bottom-right with a given total cost. At each step, it subtracts the current cell's cost and explores right and left. When the function reaches the starting cell with the remaining cost, it counts one path. The matrix is entered by the user, the same as the target cost. The recursive function explores all possible paths from the destination cell back to the starting cell, then at each step, it subtracts the cost of the current step from the remaining target cost. If the cost becomes negative, and then that path is skipped.

D. Assessment Rubric

E. External References

GeeksforGeeks. (2025a, July 23). Brute Force Approach and its pros and cons. GeeksforGeeks.
<https://www.geeksforgeeks.org/dsa/brute-force-approach-and-its-pros-and-cons/>

GeeksforGeeks. (2025c, October 25). Depth first search or DFS for a graph. GeeksforGeeks.
<https://www.geeksforgeeks.org/dsa/depth-first-search-or-dfs-for-a-graph/>

GeeksforGeeks. (2025a, July 23). Backtracking algorithm. GeeksforGeeks.
<https://www.geeksforgeeks.org/dsa/backtracking-algorithms/>

GeeksforGeeks. (2025d, September 10). Binary search. GeeksforGeeks.
<https://www.geeksforgeeks.org/dsa/binary-search/>

GeeksforGeeks. (2025e, October 22). Linear Search Algorithm. GeeksforGeeks.
<https://www.geeksforgeeks.org/dsa/linear-search/>